LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 8 ACARA 16

SEMESTER 3



Tugas :
RESTful API + endpoint
request/response

Disusun Oleh:
Uaryl Engno Laurenca (E41250317)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Pada sistem informasi akademik, data seperti mahasiswa, program studi, dan mata kuliah tidak hanya diakses lewat halaman web, tetapi sering juga dibutuhkan oleh aplikasi lain seperti aplikasi mobile atau layanan pihak ketiga. Agar data bisa diakses oleh berbagai klien tersebut, server perlu menyediakan antarmuka yang standar, yaitu API.

Pada praktikum sebelumnya, API Data Mahasiswa sudah dibuat dengan pola REST sederhana yang baru mendukung pengambilan data (GET) dan penambahan data (POST). Pada Acara 16 ini API tersebut dikembangkan menjadi RESTful API yang lengkap. API ini mendukung operasi CRUD (GET, POST, PUT, DELETE) dengan endpoint berbasis resource, format JSON, dan HTTP status code yang sesuai. Pengujiannya dilakukan memakai Postman, sehingga API bisa diuji tanpa membuat frontend terlebih dahulu.

Implementasi memakai PHP dengan PDO dan database MySQL, dan disusun berlapis (Controller, Service, Repository) mengikuti pola yang sudah dipelajari pada acara-acara sebelumnya.

B. TUJUAN PRAKTIKUM

1. Memahami konsep RESTful API, resource, endpoint, request, dan response.
2. Mengimplementasikan endpoint GET (semua data dan berdasarkan ID), POST, PUT, dan DELETE untuk resource mahasiswa.
3. Menggunakan JSON sebagai format request dan response.
4. Menerapkan HTTP status code yang sesuai (200, 201, 400, 404, 500, dan tambahan 405, 409).
5. Menguji seluruh endpoint, termasuk kondisi error, menggunakan Postman.
6. Memahami alur Endpoint + HTTP Method + Request, lalu RESTful API, Database, dan Response JSON.

C. DASAR TEORI

RESTful API

RESTful API adalah API yang menerapkan prinsip REST (Representational State Transfer) untuk menyediakan dan mengelola data melalui HTTP. Data direpresentasikan sebagai resource, misalnya mahasiswa, prodi, dosen, dan mata_kuliah. Tiap resource punya endpoint sendiri, misalnya /api/mahasiswa.

HTTP Method

Operasi terhadap resource ditentukan oleh HTTP method:

- GET : mengambil data (/api/mahasiswa, /api/mahasiswa/1)
- POST : menambahkan data (/api/mahasiswa)
- PUT : mengubah seluruh data (/api/mahasiswa/1)
- PATCH : mengubah sebagian data

- DELETE : menghapus data (/api/mahasiswa/1)

Endpoint
Endpoint adalah alamat untuk mengakses fungsi tertentu pada API, contohnya http://localhost/latihan-acara16/api/mahasiswa. Endpoint dapat berparameter, misalnya angka 1 pada /api/mahasiswa/1 menunjukkan ID mahasiswa yang diakses.
Request
Request adalah permintaan dari client ke API. Komponennya adalah HTTP method, URL/endpoint, header, parameter, dan body. Untuk mengirim data baru, client memakai header Content-Type: application/json dan body berformat JSON.

Response dan HTTP Status Code
Response adalah jawaban API kepada client, berupa body JSON dan status code:

- 200 : request berhasil
- 201 : data berhasil dibuat
- 400 : request tidak valid
- 404 : data/endpoint tidak ditemukan
- 500 : terjadi error pada server

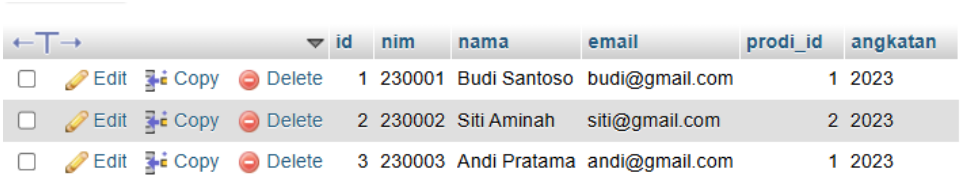
Postman
Postman adalah aplikasi untuk membuat, mengirim, dan menguji HTTP request ke API. Dengan Postman, pengujian dapat dilakukan tanpa frontend, dan seluruh request dapat disimpan sebagai Collection.

D. HASIL DAN PEMBAHASAN

Alat dan Bahan
Laptop/PC, web browser, Visual Studio Code, XAMPP/Laragon (Apache dan MySQL), PHP, Postman, dan modul praktikum.

Struktur Proyek (folder Acara16)
.htaccess : rewrite URL /api/mahasiswa/{id} ke api/mahasiswa.php?id={id}
database.sql : skema database dan data awal
config/database.php : pengaturan koneksi
api/mahasiswa.php : titik masuk resource mahasiswa
api/notfound.php : response 404 JSON untuk endpoint tidak dikenal
app/Database.php : koneksi PDO (singleton)
app/MahasiswaRepository.php : query database
app/MahasiswaService.php : validasi dan aturan bisnis
app/MahasiswaApiController.php : pengarah request berdasarkan HTTP method
app/JsonResponse.php, Exceptions.php, Logger.php : pendukung
[SCREENSHOT 1: struktur folder proyek di Visual Studio Code]

Database
Database bernama si_akademik berisi tabel mahasiswa dengan kolom id (primary key, auto increment), nim (unik), nama, email, created_at, dan updated_at. Tiga data awal dimasukkan (Budi, Sari, Andi) agar endpoint GET langsung bisa diuji.



	id	nim	nama	email	prodi_id	angkatan
Edit Copy Delete	1	230001	Budi Santoso	budi@gmail.com	1	2023
Edit Copy Delete	2	230002	Siti Aminah	siti@gmail.com	2	2023
Edit Copy Delete	3	230003	Andi Pratama	andi@gmail.com	1	2023

SCREENSHOT 2: tabel mahasiswa beserta datanya di phpMyAdmin

Implementasi

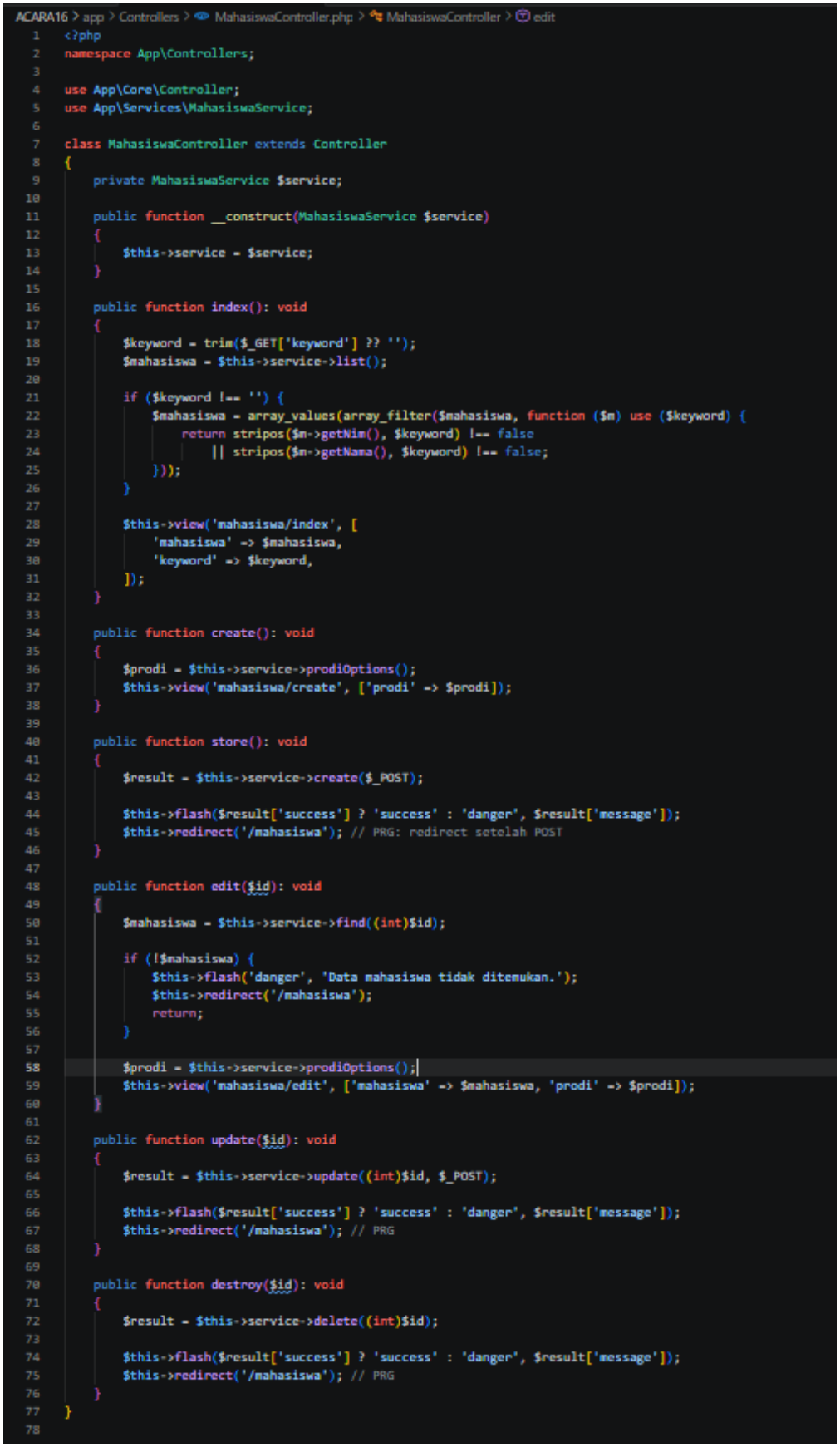
a. Routing. File .htaccess memetakan /api/mahasiswa ke api/mahasiswa.php dan /api/mahasiswa/{id} ke api/mahasiswa.php?id={id}. Dengan begitu URL tetap bersih seperti yang diminta modul, sementara kode PHP cukup membaca parameter id. URL /api/... lain yang tidak dikenal dialihkan ke notfound.php dan menghasilkan 404 berformat JSON.

b. Controller. MahasiswaController membaca HTTP method lalu memanggil aksi yang sesuai: GET (daftar atau detail), POST (tambah), PUT (ubah), DELETE (hapus). Controller juga membaca body JSON, memvalidasi ID, dan menerjemahkan exception menjadi status code (ValidationException ke 400, NotFoundException ke 404, ConflictException ke 409, error lain ke 500). Method yang tidak didukung menghasilkan 405.

c. Service. MahasiswaService berisi validasi: nim, nama, dan email wajib ada dan tidak boleh kosong, email harus berformat valid, serta NIM tidak boleh duplikat. Service juga memeriksa keberadaan data sebelum PUT dan DELETE.

d. Repository. MahasiswaRepository menjalankan query dengan prepared statement PDO (SELECT, INSERT, UPDATE, DELETE) sehingga aman dari SQL injection.

e. Penanganan error server. Detail error (misalnya kegagalan koneksi database) hanya dicatat di storage/logs/app.log, sedangkan client hanya menerima pesan umum “Terjadi kesalahan pada server” dengan status 500.



SCREENSHOT 3: kode MahasiswaController.php

ACARA16 > public > .htaccess

```
RewriteEngine On

# Rule khusus API RESTful: /api/mahasiswa/5 -> api/mahasiswa.php?id=5
RewriteCond %{REQUEST_FILENAME} !-f
RewriteRule ^api/mahasiswa/([0-9]+)/?$ api/mahasiswa.php?id=$1 [L,QSA]

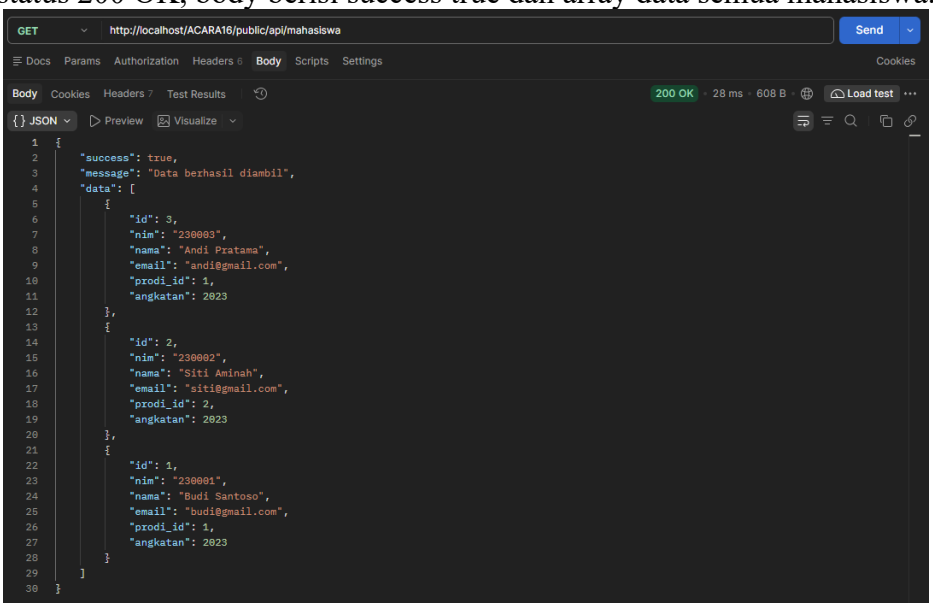
# Rule khusus API RESTful: /api/mahasiswa -> api/mahasiswa.php (GET semua / POST tambah)
RewriteCond %{REQUEST_FILENAME} !-f
RewriteRule ^api/mahasiswa/?$ api/mahasiswa.php [L,QSA]

# Rule MVC bawaan: semua request lain diarahkan ke front controller
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]

```

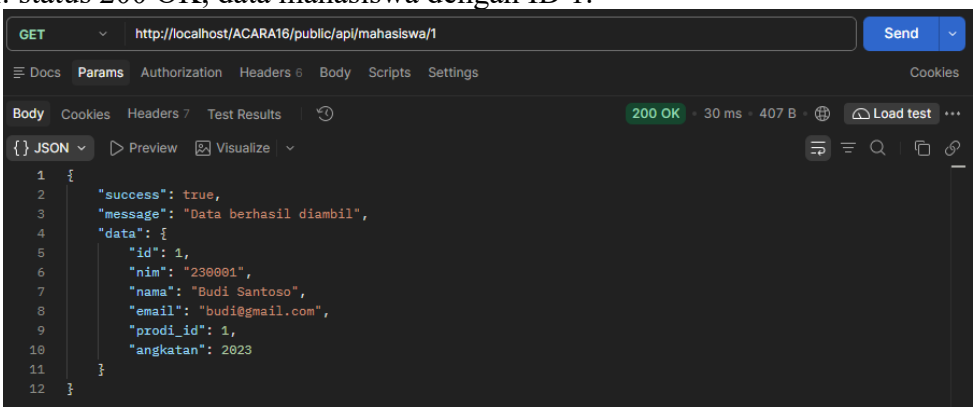
SCREENSHOT 4: isi file .htaccess

Pengujian Endpoint Berhasil dengan Postman
Base URL: http://localhost/ACARA16/public/
(1) GET semua mahasiswa
GET /api/mahasiswa
Hasil: status 200 OK, body berisi success true dan array data semua mahasiswa.



SCREENSHOT 5: Postman GET /api/mahasiswa, status 200 beserta body

(2) GET berdasarkan ID
GET /api/mahasiswa/1
Hasil: status 200 OK, data mahasiswa dengan ID 1.

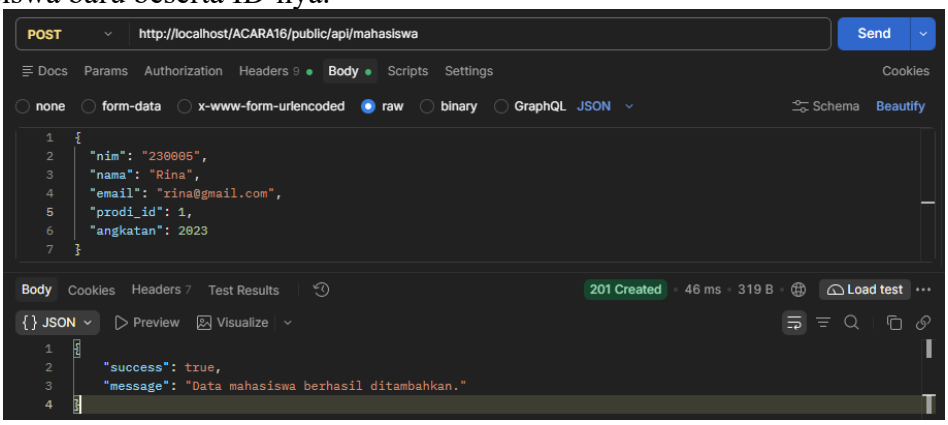


SCREENSHOT 6: Postman GET /api/mahasiswa/1, status 200

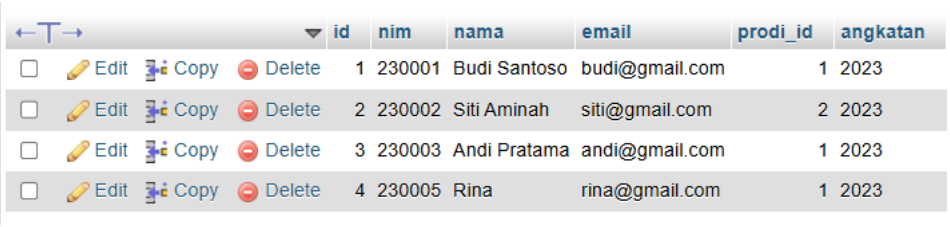
(3) POST tambah mahasiswa
POST /api/mahasiswa, Body, raw, JSON:

```
{
  "nim": "230005",
  "nama": "Rina",
  "email": "rina@gmail.com",
  "prodi_id": 1,
  "angkatan": 2023
}
```

Hasil: status 201 Created, message “Data mahasiswa berhasil ditambahkan”, dan data mahasiswa baru beserta ID-nya.



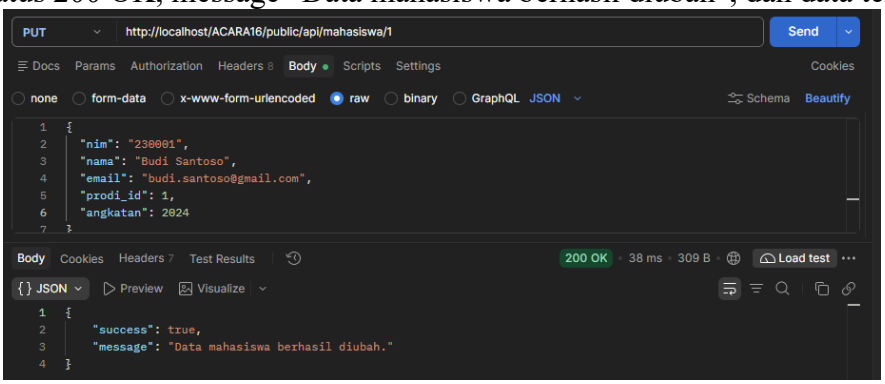
SCREENSHOT 7: Postman POST, body request dan response 201



	id	nim	nama	email	prodi_id	angkatan
Edit Copy Delete	1	230001	Budi Santoso	budi@gmail.com	1	2023
Edit Copy Delete	2	230002	Siti Aminah	siti@gmail.com	2	2023
Edit Copy Delete	3	230003	Andi Pratama	andi@gmail.com	1	2023
Edit Copy Delete	4	230005	Rina	rina@gmail.com	1	2023

SCREENSHOT 8: phpMyAdmin menunjukkan data Rina masuk ke tabe

(4) PUT ubah mahasiswa
PUT /api/mahasiswa/1, Body:
{ “nim”: “23001”, “nama”: “Budi Santoso”, “email”: “budi.santoso@gmail.com” }
Hasil: status 200 OK, message “Data mahasiswa berhasil diubah”, dan data terbaru.

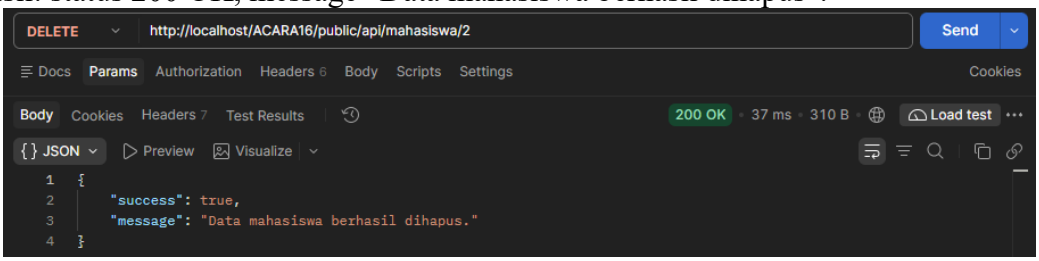


SCREENSHOT 9: Postman PUT, body request dan response 200

	id	nim	nama	email	prodi_id	angkatan
Edit Copy Delete	1	230001	Budi Santoso	budi.santoso@gmail.com	1	2024
Edit Copy Delete	2	230002	Siti Aminah	siti@gmail.com	2	2023
Edit Copy Delete	3	230003	Andi Pratama	andi@gmail.com	1	2023
Edit Copy Delete	4	230005	Rina	rina@gmail.com	1	2023

SCREENSHOT 10: phpMyAdmin menunjukkan data ID 1 sudah berubah

(5) DELETE hapus mahasiswa
DELETE /api/mahasiswa/2
Hasil: status 200 OK, message "Data mahasiswa berhasil dihapus".

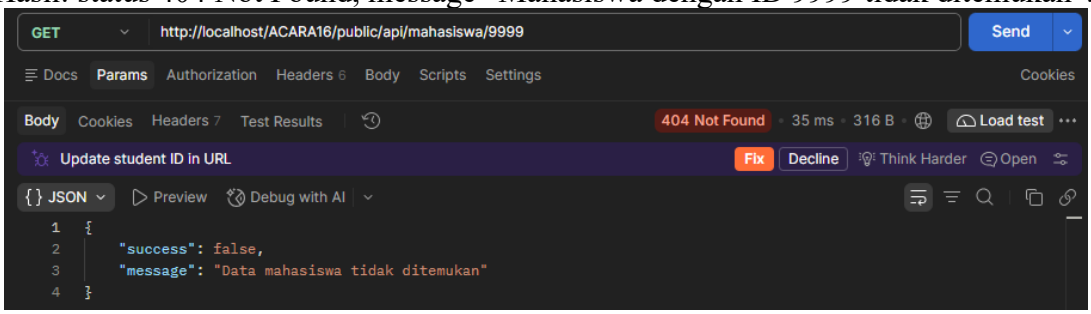


SCREENSHOT 11: Postman DELETE, status 200

	id	nim	nama	email	prodi_id	angkatan
Edit Copy Delete	1	230001	Budi Santoso	budi.santoso@gmail.com	1	2024
Edit Copy Delete	3	230003	Andi Pratama	andi@gmail.com	1	2023
Edit Copy Delete	4	230005	Rina	rina@gmail.com	1	2023

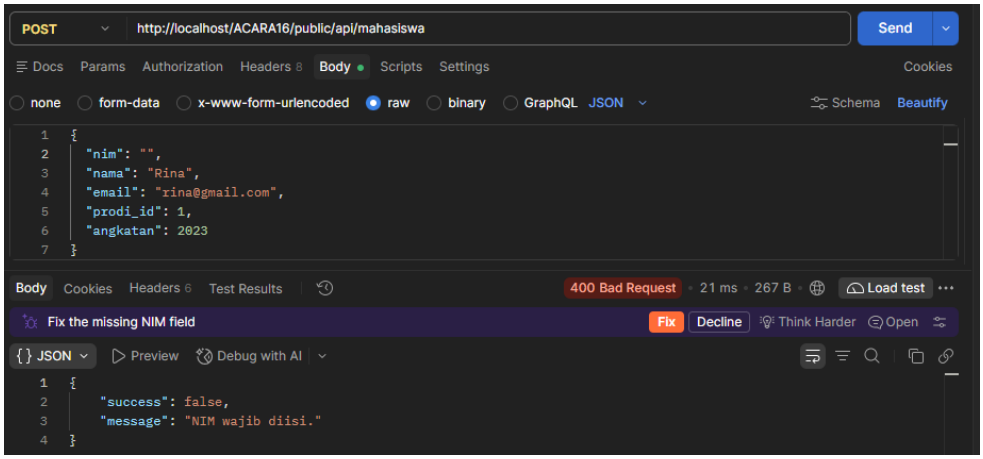
SCREENSHOT 12: phpMyAdmin menunjukkan data ID 2 sudah tidak ada

4.6 Pengujian Kondisi Error
(1) ID tidak ditemukan
GET /api/mahasiswa/9999
Hasil: status 404 Not Found, message "Mahasiswa dengan ID 9999 tidak ditemukan".



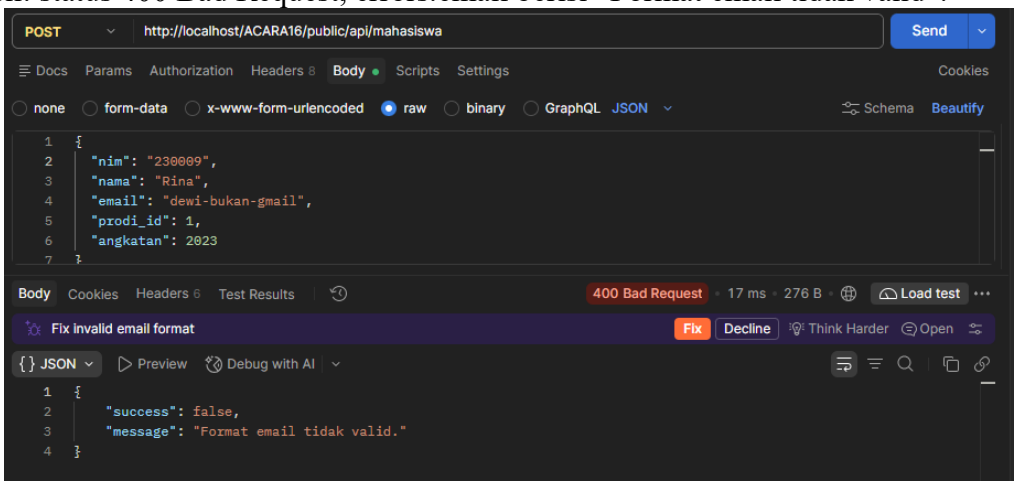
SCREENSHOT 13: Postman GET ID 9999, status 404

(2) NIM kosong
POST /api/mahasiswa dengan nim "" (kosong).
Hasil: status 400 Bad Request, errors.nim berisi "NIM tidak boleh kosong".



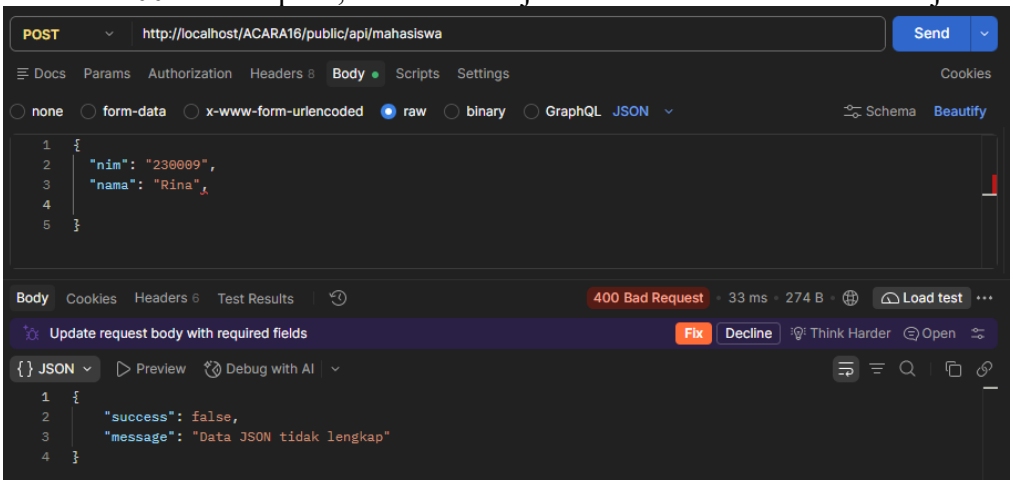
SCREENSHOT 14: Postman POST NIM kosong, status 400

(3) Email tidak valid
POST /api/mahasiswa dengan email "dewi-bukan-email".
Hasil: status 400 Bad Request, errors.email berisi "Format email tidak valid".



SCREENSHOT 15: Postman POST email tidak valid, status 400

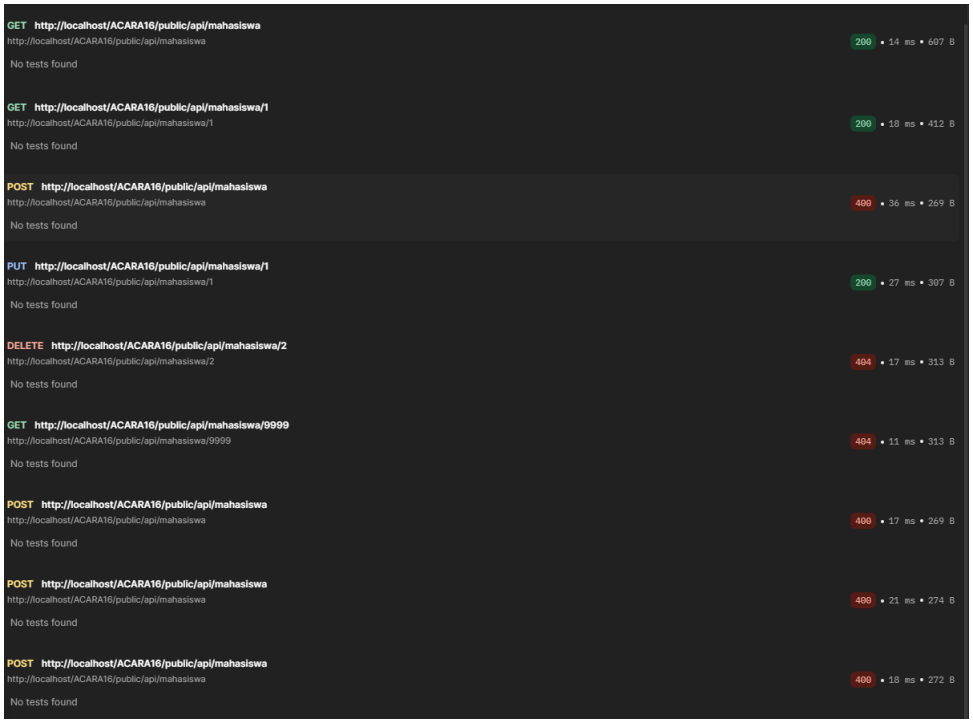
(4) Data JSON tidak lengkap
POST /api/mahasiswa dengan body hanya { "nim": "23004" }.
Hasil: status 400 Bad Request, errors menunjukkan field nama dan email wajib ada.



SCREENSHOT 16: Postman POST JSON tidak lengkap, status 400

Pengujian dengan Collection Postman

Seluruh request disimpan dalam satu Collection (“Acara 16 - RESTful API Mahasiswa”) yang terbagi menjadi folder “Berhasil” dan “Error”. Tiap request memiliki test otomatis yang memeriksa status code dan format JSON. Collection dijalankan lewat Collection Runner.



SCREENSHOT 18: hasil Collection Runner

Pembahasan

1. Penggunaan resource dan HTTP method membuat endpoint tetap sederhana. Satu URL /api/mahasiswa dipakai untuk daftar dan tambah data, sedangkan /api/mahasiswa/{id} dipakai untuk detail, ubah, dan hapus. Perbedaannya hanya pada HTTP method.
2. Status code membuat client mudah membedakan hasil request tanpa membaca isi pesan: 201 untuk data baru, 404 untuk data tidak ada, dan 400 untuk input yang salah. Kode 409 dipakai untuk NIM duplikat karena masalahnya bukan format input, melainkan konflik dengan data yang sudah ada.
3. Pemisahan Controller, Service, dan Repository membuat tanggung jawab jelas: Controller mengurus HTTP, Service mengurus validasi dan aturan bisnis, Repository mengurus database. Perubahan aturan validasi cukup dilakukan di Service tanpa menyentuh Controller.
4. Validasi dilakukan di sisi server karena data dari client tidak bisa dipercaya. Prepared statement mencegah SQL injection, dan pesan error server disembunyikan dari client demi keamanan.
5. PUT pada praktikum ini bersifat mengganti seluruh data, sehingga nim, nama, dan email harus dikirim lengkap. Untuk ubah sebagian data diperlukan PATCH, yang belum diimplementasikan karena tidak diminta modul.
6. Postman terbukti memudahkan pengujian karena request, body JSON, dan hasil test otomatis bisa disimpan dan dijalankan ulang kapan saja tanpa frontend.

E. KESIMPULAN

RESTful API Data Mahasiswa berhasil dibuat dengan lima endpoint, yaitu GET untuk mengambil seluruh data mahasiswa, GET berdasarkan ID, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data. API menggunakan format JSON untuk request dan response serta menerapkan HTTP status code yang sesuai, seperti 200, 201, 400, 404, 405, 409, dan 500. Seluruh endpoint beserta kondisi error, seperti ID tidak ditemukan, NIM kosong, email tidak valid, dan JSON tidak lengkap, telah berhasil diuji menggunakan Postman dengan hasil yang sesuai rancangan. Alur kerja yang diterapkan meliputi Endpoint, HTTP Method, Request, RESTful API, Database, hingga Response JSON, dengan Postman sebagai alat untuk melakukan verifikasi. Selain itu, penerapan struktur berlapis yang terdiri dari Controller, Service, dan Repository membuat kode program lebih terstruktur, mudah dipelihara, serta dapat dikembangkan untuk mengelola resource lain, seperti program studi, dosen, dan mata kuliah.